

MOSES C++ Technical Specification

Elias Lindstenz

February 2025

Contents

1	Introduction	3
2	Initialisation	4
3	Main Simulation	6
4	Yearly Functions	7
4.1	Yearly Initialisation	7
4.2	Yearly Expectations and Target	8
4.3	Yearly Update	9
5	Quarterly Functions	10
5.1	Quarterly Expectations	10
5.2	Production Planning	11
5.2.1	Labour Unemployment Update	11
5.2.2	Production Frontier	12
5.2.3	Initialise Production Plan	12
5.2.4	Target Search	13
5.3	Labour Market	15
5.3.1	Government Labour	15
5.3.2	Industrial Labour	15
5.4	Export Market	18
5.5	Domestic Market	19
5.5.1	Market Entrance	19
5.5.2	Household Init	19
5.5.3	Market Confront	20
5.5.4	Compute Imports	21
5.5.5	Domestic Results	22
5.5.6	External sectors	23
5.5.7	Household Update	23
5.5.8	Indirect Taxes	24
5.6	Inventory System	25
5.7	Final Quarterly Prices, Sales, and Margin	27
5.8	Quarterly Cumulation	28
5.9	Investment and Financing	29
5.10	Government Accounting	31
5.11	Monetary Sector	33
5.12	National Accounting	35
6	Helper Functions	37
7	Variable List	42

1 Introduction

The following document contains the pseudocode for the C++ version of the Model Of the Swedish Economic System (MOSES) as of February 2025. The project was first designed in A Programming Language (APL). In 2023, it was converted to a preliminary C++ model based on the preliminary pseudocode from 1976. In 2025, the updated APL project was converted with the help of the pseudocode from 1989. The following pseudocode is largely equivalent to the pseudocode from 1989. There are minor changes that replicate the most updated APL version instead of the pseudocode. The pseudocode from 1989 has small differences to the current APL project. This pseudocode and the C++ model should be regarded as a hybrid between the two.

This pseudocode has two purposes. First, it adjusts the differences from the older pseudocode. Second, it replicates C++'s object-oriented structure and style, which increases readability.

In object-oriented coding, classes are templates of objects. For example, the Firm class describes the properties of a firm. The Firm class may contain variables such as margin, price, and sales. Variables refer locally to their respective class instances. In other words, the sales variable s is not a globally defined array of all sales. It is the sales of a specific company. Classes also contain functions, called methods. These specify functionality of individual class instances. To initialise the production plan of a particular firm, for example, a firm calls *Firm :: init_prodplan*.

Furthermore, the C++ implementation has polymorphic characteristics. Explicitly producing sectors (Market) and externals sectors (External) share functionality. They are both sectors, but differ in significant ways. This is programmatically implemented by classifying the Market and External classes as children of the Sector class. In other words, the classes inherit from their base class Sector. They are types of the Sector class. All markets and externals have copies of all Sector variables, but Market instances do not have copies of External-specific variables and vice versa. All sectors have an id, but only externals have the x variable, and only markets have the io variable, for example. The same rule applies to methods. For instance, both classes have their own variant of the *quarterly_exp* function, which is first declared in the Sector class, and later implemented in the Market and External classes for their respective specific differences. Market-specific and External-specific methods are only part of their respective class.

The MOSES model is a very tightly coupled system. Different classes rely heavily on each other for functionality. The input-output coefficients for firms in a market are stored in the Market class, and not in the Firm class for instance. Therefore, firms must access their respective Market instances for access to that variable. The downside of such a design is that the pseudocode may become ambiguous, especially since multiple classes can have the same variable name. For example, the Household and the Government classes have the variable *qint*. To remove ambiguity, assume that variables within methods refer to that class unless specified otherwise. The same applies to loops. For example, if a calculation is computed for every firm, it should first be assumed that the calculation refers to the firms' variables. In more complicated cases, ambiguity is removed by adding a note under the function, or by prefixing the variable with the correct class. For instance, in a calculation involving both government and household receipts, the variables would appear like this: *Government :: qint*, *Household :: qint*.

It should be noted that C++ is a 0-indexed programming language. The first item of an array is accessed with a zero index, not the number one. Therefore, the index-variable for the durables sector is two instead of three for example.

The structure of the model can always be improved. It is difficult to achieve perfectly consistent naming since C++-specific terms limit naming. For example, government interest receipts on a yearly basis are named *intg* instead of *int* since *int* refers to the integer datatype.

In addition, variable classification can be discussed. For instance, it can be argued that household

tax variables should be in the Household class since they refer to households. On the other hand, one may argue that all tax variables should be contained in the Government class since the government regulates taxes.

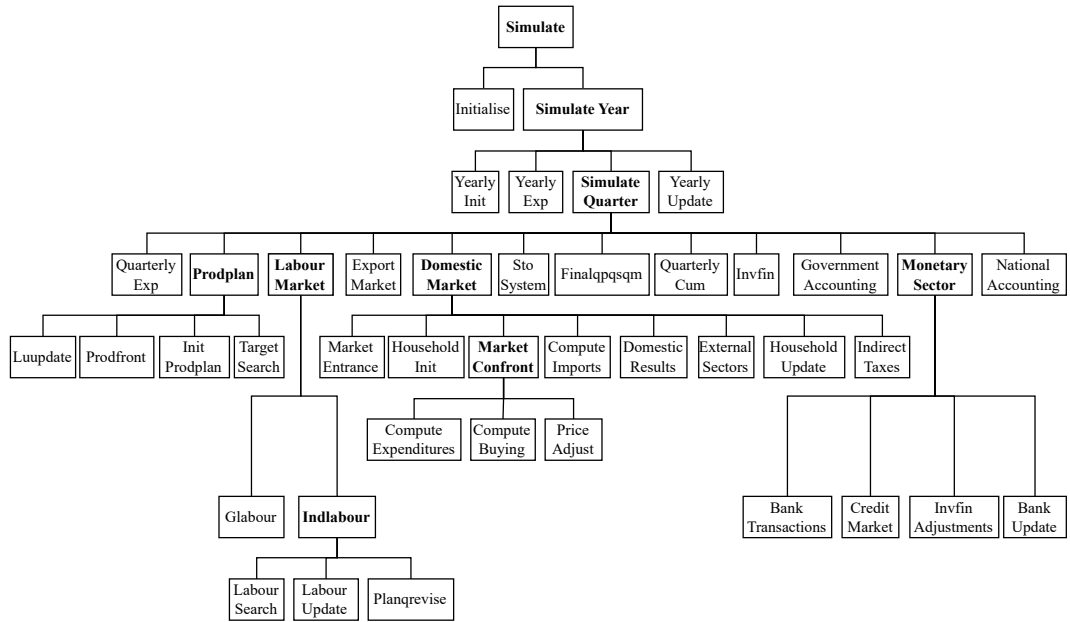
The complete MOSES simulation is overviewed in Figure 1. The diagram is read from top to bottom and from left to right. Each module represents an important function. The simulation starts in *Simulate*, enters *Initialise* before *Simulate Year*, and completes *Prodplan*'s sub-modules before moving on to *Labour Market*.

The project source code is located in the public Github repository: github.com/novrion/MOSES.

2 Initialisation

Initialisation is uncomplicated. Input data necessary for functional simulation is stored in csv files. Reference the source code and the csv files for a template of how to fill the csv files with input data. A simple parser converts the data from the csv files into data within object instances before simulation.

Figure 1: A Block Diagram of MOSES



3 Main Simulation

Algorithm 1 Simulate

```
function SIMULATE
  INITIALISE
  for all years do
    SIMULATE_YEAR
    year = year + 1
```

Algorithm 2 Simulate Year

```
function SIMULATE_YEAR
  YEARLY_INIT
  for all firms do
    YEARLY_EXP
    YEARLY_TARG
  for relative_quarter = 0, 1, ..., 3 do
    SIMULATE_QUARTER
    quarter = quarter + 1
  for all firms do
    FIRM::YEARLY_UPDATE
```

Algorithm 3 Simulate Quarter

```
function SIMULATE_QUARTER
  QUARTERLY_EXP
  PRODPLAN
  LABOUR_MARKET
  EXPORT_MARKET
  DOMESTIC_MARKET
  for all firms do
    FINALQPQSQM
    QUARTERLY_CUM
  INVFIN
  GOVERNMENT_ACCOUNTING
  NATIONAL_ACCOUNTING
```

4 Yearly Functions

4.1 Yearly Initialisation

Algorithm 4 Yearly Init

```
function YEARLY_INIT
    cumgnpcur = 0
    cumgnpfix = 0
    cumexport = 0
    cumimport = 0

    GOVERNMENT::YEARLY_INIT
for all firms do
    FIRM::YEARLY_INIT
```

Algorithm 5 Yearly Init (Government)

```
function GOVERNMENT::YEARLY_INIT
    cumwtax = 0
    cumitax = 0
    cumvatax = 0
    cumctax = 0
    cumws = 0
    cuml = 0
    cuminv = 0
    cumpurch = 0
    cumtrans = 0
    cumsubs = 0
    cummprint = 0
    cumint = 0
```

Algorithm 6 Yearly Init (Firm)

```
function FIRM::YEARLY_INIT
    cumq = 0
    cumm = 0
    cumsu = 0
    cums = 0
    cump = 0
    cumws = 0
    cuml = 0
    cuminv = 0
    cumva = 0
    cumsnet = 0
```

4.2 Yearly Expectations and Target

Algorithm 7 Yearly Exp

function FIRM::YEARLY_EXP

```
histdp = smp * histdp + (1 - smp) * dp
histdpdev = smp * histdpdev + (1 - smp) * (dp - expdp)
histdpdev2 = smp * histdpdev2 + (1 - smp) * (dp - expdp)2
expidp = histdp + e1 * histdpdev - e2 *  $\sqrt{\text{histdpdev2}}$ 
expdp = (1 - r) * expidp + r * expdp

histdw = smw * histdw + (1 - smw) * dw
histdwdev = smw * histdwdev + (1 - smw) * (dw - expdw)
histdwdev2 = smw * histdwdev2 + (1 - smw) * (dw - expdw)2
expidw = histdw + e1 * histdwdev + e2 *  $\sqrt{\text{histdwdev2}}$ 
expdw = (1 - r) * expidw + r * expdw

histds = sms * histds + (1 - sms) * ds
histdsdev = sms * histdsdev + (1 - sms) * (ds - expds)
histdsdev2 = sms * histdsdev2 + (1 - sms) * (ds - expds)2
expids = histds + e1 * histdsdev - e2 *  $\sqrt{\text{histdsdev2}}$ 
expds = (1 - r) * expids + r * expds
```

Algorithm 8 Yearly Targ

function FIRM::YEARLY_TARG

```
mhist = MAX(0, smt * mhist + (1 - smt) * m)
targm = mhist * (1 + eps)
```

4.3 Yearly Update

Algorithm 9 Yearly Update

function FIRM::YEARLY_UPDATE

$dq = \frac{cumq}{q} - 1$
 $q = q * (1 + dq)$
 $dp = \frac{cums}{cumsu * p} - 1$
 $p = p * (1 + dp)$
 $dw = \frac{cumws}{cuml * w} - 1$
 $w = w * (1 + dw)$
 $ds = \frac{cums}{s} - 1$
 $s = s * (1 + ds)$
 $dva = \frac{cumva}{va} - 1$
 $va = va * (1 + dva)$
 $snet = cumsnet$
 $chm = cumm - m$
 $m = m + chm$

5 Quarterly Functions

5.1 Quarterly Expectations

Algorithm 10 Quarterly Exp

```
function QUARTERLY_EXP
  for all firms do
    FIRM::QUARTERLY_EXP
    FIRM::QUARTERLY_TARG
  for all markets do
    MARKET::QUARTERLY_EXP
  for all externals do
    EXTERNAL::QUARTERLY_EXP
```

Algorithm 11 Quarterly Exp (Firm)

```
function FIRM::QUARTERLY_EXP
   $qexpdp = \frac{expdp}{4}$ 
   $qexpdw = \frac{expdw}{4}$ 
   $qexpds = \frac{expds}{4}$ 
  if last quarter in year then
     $qexpdp = qexpdp + fip * (qdp - qexpdp)$ 
     $qexpdw = qexpdw + fiw * (qdw - qexpdw)$ 
     $qexpds = qexpds + fis * (qds - qexpds)$ 
   $qexpp = qp * (1 + qexpdp)$ 
   $qexpw = qw * (1 + qexpdw)$ 
   $qexps = qs * (1 + qexpds)$ 
```

Algorithm 12 Quarterly Targ

```
function FIRM::QUARTERLY_TARG
   $qtargm = targm$ 
```

Algorithm 13 Quarterly Exp (Market)

```
function MARKET::QUARTERLY_EXP
   $qexpdpim = \frac{SUM(qq * qexpdp)}{SUM(qq)}$ 
   $qexppim = (1 - txva2) * qpdome * (1 + qexpdpim)$ 
```

Algorithm 14 Quarterly Exp (External)

```
function EXTERNAL::QUARTERLY_EXP
   $qexpdpim = qdp - qchtxva2$ 
   $qexppim = (1 - txva2) * qpdome * (1 + qexpdpim)$ 
```

5.2 Production Planning

Algorithm 15 Prodplan

```
function PRODPLAN
  LUUPDATE
  PRODFRONT
  for all firms do
    FIRM::INIT_PRODPLAN
    if not FIRM::TARGET_SEARCH then
      NULLIFY_FIRM

  if no firms left in a market then
    exit simulation

  for all firms do
    qplanqsave = qplanq
```

5.2.1 Labour Unemployment Update

Algorithm 16 luupdate

```
function LUUPDATE
  LABOUR::LUUPDATE
  for all firms do
    FIRM::LUUPDATE
```

Algorithm 17 luupdate (Labour)

```
function LABOUR::LUUPDATE
   $lf = lu + \text{Government} :: l + \text{SUM}(\text{Firm} :: l)$ 
   $lu = lu * (1 - ret)$ 
   $lu = lu + entry * lf$ 
```

Algorithm 18 Luupdate (Firm)

```
function FIRM::LUUPDATE
   $l = l * (1 - ret)$ 
  for  $i = 0, 1, 2$  do
     $aman[i] = aman[i] * (1 - ret)$ 
```

5.2.2 Production Frontier

Algorithm 19 Prodfont

```
function PRODFRONT
  for all markets do
    mtec = mtec * (1 + qdmtec)
  for all firms do
    FIRM::PRODFRONT
```

Algorithm 20 Prodfont (Firm)

```
function FIRM::PRODFRONT
  qtop = qtop * (1 - rho)
  qchqtop1 = (1 - loss) *  $\frac{qinv*inveff}{qp}$ 
  qchqtop2 = MIN(loss *  $\frac{qinv*inveff}{qp}$  *  $\frac{resmax-res}{resmax}$ , (qtop + qchqtop1) *  $\frac{resmax-res}{1-resmax}$ )
  qchqtop = qchqtop1 + qchqtop2

  res =  $\frac{res*(qtop+qchqtop1)+qchqtop2}{qtop+qchqtop}$ 
  if res < 0 or res > resmax then
    exit simulation

  tec =  $\frac{qtop+qchqtop}{\frac{qtop}{tec} + \frac{qchqtop}{mtec}}$ 
  qtop = qtop + qchqtop
```

5.2.3 Initialise Production Plan

Algorithm 21 Init Prodplan

```
function FIRM::INIT_PRODPLAN
  qexpsu =  $\frac{qexps}{qexpp}$ 
  qplanq = MAX(0, qexpsu +  $\frac{optsto-sto}{4*tmsto}$ )
```

5.2.4 Target Search

Algorithm 22 Target Search

```
function FIRM::TARGET_SEARCH
     $qexpnet = qexp - share * (io \times (qpdome * (1 - txva2)))$ 

    if  $qplanq \geq qtop * (1 - res) * wtix$  then
        return PLAN_IMPLIES_RECRUITMENT

    if  $qplanq > QFR(l)$  then
        if SAT( $qplanq$ , RFQ( $qplanq$ )) then
             $qplanl = RFQ(qplanq)$ 
            HANDLE_LAYOFFS
            return true
        return PLAN_IMPLIES_RECRUITMENT

    if SAT( $qplanq$ ,  $l$ ) then
         $qplanl = l$ 
        HANDLE_LAYOFFS
        return true

    return TRY_INCREASE_PRODUCTION
```

The firm's market's io is matrix multiplied with market prices.

Algorithm 23 Plan Implies Recruitment

```
function FIRM::PLAN_IMPLIES_RECRUITMENT
    if SAT(QFR( $l$ ),  $l$ ) then
        SOLVE
        HANDLE_LAYOFFS
        return true
    return TRY_REDUCE_SLACK(QFR( $L$ ))
```

Algorithm 24 Try Reduce Production

```
function FIRM::TRY_REDUCE_PRODUCTION
    if SAT( $qplanq$ , RFQ( $qplanq$ )) then
        SOLVE
        HANDLE_LAYOFFS
        return true
    return TRY_REDUCE_SLACK( $qplanq$ )
```

Algorithm 25 Try Increase Production

function FIRM::TRY_INCREASE_PRODUCTION $q2 = \text{MIN}(\text{QFR}(l), qexpsu + maxsto - sto)$ **if** SAT($q2, 1$) **then** $qplanq = \frac{l * \frac{qexpw}{4}}{(1 - qtargm) * qexpnet}$
 $qplanl = l$

HANDLE_LAYOFFS

return true**if** $q2 = \text{QFR}(l)$ **then****return** TRY_REDUCE_PRODUCTION**if** SAT($q2, \text{RFQ}(q2)$) **then** $qplanq = q2$
 $qplanl = \frac{(1 - qtargm) * q2 * qexpnet}{\frac{qexpw}{4}}$

HANDLE_LAYOFFS

return true**return** TRY_REDUCE_PRODUCTION

Algorithm 26 Try Reduce Slack

function FIRM::TRY_REDUCE_SLACK($q7$)**if** SAT($q7, \text{RFQ}(\frac{1 - res}{1 - resdown * res} * q7)$) **then** $qplanq = q7$
 $qplanl = \frac{(1 - qtargm) * q7 * qexpnet}{\frac{qexpw}{4}}$ $res = 1 - \frac{q7 * (1 - res)}{\text{QFR}(qplanl)}$

HANDLE_LAYOFFS

return true $res = res * resdown$ **if** SAT(0, 0) **then**

SOLVE

HANDLE_LAYOFFS

return true**return** false

If no production plan can be found, false is returned and the firm is nullified.

Algorithm 27 Handle Layoffs

function FIRM::HANDLE_LAYOFFS $layoff = \text{MAX}(0, l - qplanl)$ $aman[0] = \text{MIN}(layoff, aman[1])$ $aman[1] = \text{MIN}(layoff - aman[0], aman[2])$ $aman[2] = layoff - aman[0] - aman[1]$

5.3 Labour Market

Algorithm 28 Labour Market

```
function LABOUR_MARKET
    GLABOUR
    INDLABOUR
    for all firms do
        PLANQREVISE
```

5.3.1 Government Labour

Algorithm 29 Glabour

```
function GOVERNMENT::GLABOUR
     $qchl = \text{MIN}(lu, l * ret + qrealchl)$ 
     $l = l + qchl - ret * l$ 
     $qdw = qdwind$ 
     $qw = qw * (1 + qdw)$ 
```

5.3.2 Industrial Labour

Algorithm 30 Indlabour

```
function INDLABOUR
    LABOUR_SEARCH
    LABOUR_UPDATE
```

Algorithm 31 Labour Search

```
function LABOUR_SEARCH
    for all firms do
         $ww = qw + \text{iota} * (qexpw - qw)$ 
         $ll = l$ 
        if  $chm > 0$  then
             $chl = qplanl - l$ 
        else
             $chl = \text{RFQ}(qplanq) - l$ 
         $ll = lu$ 

    Sort firms in descending order of  $\frac{chl}{ll}$ 
    for  $iter = 0, 1, \dots, niter - 1$  do
        for all firms do
            if  $chl \leq 0$  then
                go to next firm
             $i = \text{thisfirm}$ 
             $target = \text{CHOOSE\_LABOUR\_TARGET}$ 
            if target is the unemployment pool then
                HANDLE_SUCCESSFUL_ATTACK
```

```

    else
        if  $ww[i] > ww[target] * (1 - \gamma)$  then
            HANDLE_SUCCESSFUL_ATTACK
        else
             $ww[i] = ww[i] + ksi * (ww[target] * (1 + \gamma) - ww[i])$ 

    for all firms do
         $qchl = ll - l$ 
         $qchw = ww - qw$ 
         $lu = ll$ 

```

```

    for all firms do
        HANDLE_LABOUR_SEARCH_LAYOFFS

```

chl , ww , and ll are temporary vectors. ww and chl store firms' wage and desired change in labour. ll stores firms' labour, concatenated to the unemployment pool.

Algorithm 32 Handle Successful Attack

```

function HANDLE_SUCCESSFUL_ATTACK
     $now = \min(\theta * ll[target], chl[i])$ 
     $ll[i] = ll[i] + now$ 
     $chl[i] = chl[i] - now$ 
     $ll[target] = ll[target] - now$ 
    if  $target$  is a firm then
         $chl[target] = chl[target] + now$ 

```

Algorithm 33 Handle Labour Search Layoffs

```

function FIRM::HANDLE_LABOUR_SEARCH_LAYOFFS
     $exit = \max(0, -qchl)$ 
    if  $exit > aman[0] + aman[1]$  then
         $aman[2] = \max(0, aman[2] - (exit - aman[0] - aman[1]))$ 
    if  $exit > aman[0]$  then
         $aman[1] = \max(0, aman[1] - (exit - aman[0]))$ 
    if  $exit > 0$  then
         $aman[0] = \max(0, aman[0] - exit)$ 

```

Algorithm 34 Labour Update

function LABOUR_UPDATE**for all** firms **do** $sack = \min(aman[0], \max(0, l + qchl - qplanl))$ $qchl = qchl - sack$ $aman[0] = aman[0] - sack$ $lu = lu + sack$

$$qdwind = \frac{\frac{\text{SUM}((l+qchl)*(qw+qchw))}{\text{SUM}(l+qchl)}}{\frac{\text{SUM}(l*qw)}{\text{SUM}(l)} - 1}$$

for all firms **do** $l = l + qchl$ $qdw = \frac{qchw}{qw}$ $qw = qw + qchw$

$$qchru = \frac{lu}{(lu + \text{Government}::l + \text{SUM}(\text{Firm}::l)) - ru}$$

 $ru = ru + qchru$ qdwind sums across firms.

Algorithm 35 Planqrevise

function FIRM::PLANQREVISE $qplanq = \min(qplanq, \text{QFR}(l))$ $qdq = \frac{qplanq}{qq} - 1$ $qq = qq * (1 + qdq)$ **for** $i = 0, 1, \dots, NSEC - 1$ **do** $qimq[i] = \max(0, share * io[i] * qplanqsave + \frac{optimsto[i] - imsto[i]}{4 * tmimsto})$ **for** $i = 0, 1, \dots, NSEC - 1$ **do** $imsto[i] = \min(maximsto[i], imsto[i] + qimq[i] - share * io[i] * qq)$

$$qtopsu = \max(0, qq * \frac{qcxpsu}{\frac{qcxpsu + (optsto - sto)}{4 * tmsto}})$$

5.4 Export Market

Algorithm 36 Export Market

```
function EXPORT_MARKET
  for all firms do
    FIRM::ADJUST_FOREIGN

  for all markets do
     $qpfor = qpfor * (1 + qdpfor)$ 

  for all firms do
     $qsfor = qsfor * qpfor * (1 + rsubs)$ 

   $qsubsfor = \text{SUM}(qsfor * qpfor * rsubs)$ 
   $qexport = \text{SUM}(\frac{qsfor}{1+rsubs})$ 
```

Algorithm 37 Adjust Foreign

```
function FIRM::ADJUST_FOREIGN
  if  $qpdom * (1 - txva2) > qpfor$  then
     $x = x - \frac{1}{4*tmx} * \frac{qpdom*(1-txva2)-qpfor}{qpfor}$ 
  else
     $x = x + \frac{1-x}{4*tmx} * \frac{qpfor-qpdom*(1-txva2)}{qpdom*(1-txva2)}$ 

   $qsfor = x * qoptsu$ 
```

5.5 Domestic Market

Algorithm 38 Domestic Market

```

function DOMESTIC_MARKET
    MARKET_ENTRANCE
    HOUSEHOLD::INIT
    MARKET_CONFRONT
    COMPUTE_IMPORTS
    DOMESTIC_RESULTS
    EXTERNAL_SECTORS
    HOUSEHOLD_UPDATE
    INDIRECT_TAXES

```

5.5.1 Market Entrance

Algorithm 39 Market Entrance

```

function MARKET_ENTRANCE
    for all firms do
         $qoptsudom = (1 - x) * qoptsu$ 
    for all markets do
         $qprelpdom = qpdom * (1 + qchtxva2) * \frac{\text{SUM}(qoptsudom * \frac{qcxpp}{qp})}{\text{SUM}(qoptsudom)}$ 

```

Sum across firms per market.

5.5.2 Household Init

Algorithm 40 Init (Household)

```

function HOUSEHOLD::INIT
     $qtrans = (rtrans * qttax) + lu * rlu * \frac{wstx}{4 * \text{SUM}(Firm::l)}$ 
     $qint = rih * \frac{wh}{4} * nh$ 
     $qtws = Government :: l * \frac{Government::qw}{4} + qinpay + \text{SUM}(Firm :: l * \frac{Firm::qw}{4})$ 
     $qwtax = (Government :: l * \frac{Government::qw}{4}) * \frac{txwg}{1+txwg} + (qinpay + \text{SUM}(Firm :: l * \frac{Firm::qw}{4})) * \frac{txw}{1+txw}$ 
     $qti = (qtws - qwtax) + qint + qtrans + qtdiv$ 
     $qitax = qti * txi1$ 
     $qdi = \frac{qti - qitax}{nh}$ 
     $qspsavreq = \frac{qsavhreq}{nh}$ 

```

5.5.3 Market Confront

Algorithm 41 Market Confront

```

function MARKET_CONFRONT
  for all markets do
    if  $qpd\text{om} > qp\text{for}$  then
       $imp = imp + \frac{1-imp}{4*tmimp} * \frac{qpd\text{om}*(1-txva2)-qp\text{for}}{qp\text{for}}$ 
    else
       $imp = imp - \frac{imp}{4*tmimp} * \frac{qp\text{for}-qpd\text{om}*(1-txva2)}{qpd\text{om}*(1-txva2)}$ 
       $pt = (1 - imp) * qp\text{rel}p\text{dom} + imp * \frac{qp\text{for}}{1-txva2}$ 
  for all externals do
     $pt = qpd\text{om} * (1 + qdp)$ 
  for  $i = 0, 1, \dots, NSEC - 1$  do
     $qpurch[i] = gkoff[i] * Government :: l * \frac{Government::qw}{4} * pt[i] * \frac{wgre\text{f}}{100*Government::qw}$ 

  for  $iter = 0, 1, \dots, marketiter - 1$  do
    COMPUTE_EXPENDITURES
    COMPUTE_BUYING
    if  $iter = marketiter - 1$  then
      for all markets do
        PRICE_ADJUST

```

Clamp imp between 0 and 1.

Algorithm 42 Compute Expenditures

```

function HOUSEHOLD::COMPUTE_EXPENDITURES
   $qp\text{rel}cpi = \frac{SUM(qc)}{SUM(\frac{qc}{pt})}$ 
   $qchdcpi = \frac{qp\text{rel}cpi}{qcpi} - 1 - qdcpi$ 

  for all  $i = 0, 1, \dots, NEXPH - 1$  do
    if  $i \neq DUR$  and  $i \neq SAV$  then
       $qspe[i] = cva[i] * pt[i]$ 

   $swap = alfa3 * (\frac{qchri}{4} - qchdcpi) + alfa4 * qchru$ 
   $qspe[DUR] = \text{MAX}(0, \frac{pt[DUR]*cva[DUR]}{rhodur} - \frac{pt[DUR]}{qp[DUR]} * (1 - rhodur) * stodur - qdi * swap)$ 
   $qspe[SAV] = (whra * qdi - wh) + qdi * swap$ 

  for  $i = 0, 1, \dots, NEXPH - 1$  do
     $qsp[i] = beta1[i] * qspe[i] + (beta2[i] + \frac{beta3[i]*qp\text{rel}cpi}{qdi-qsp\text{sav}req}) * (qdi - qsp\text{sav}req - SUM(qspe[i] * beta1[i]))$ 
    if  $i \neq SAV$  then
       $qsp[i] = \text{MAX}(0, qsp[i])$ 
   $qsp[SAV] = qsp[SAV] + qsp\text{sav}req$ 

```

Clamp swap between -0.05 and -0.05

Algorithm 43 Compute Buying

```
function COMPUTE_BUYING
  for all markets do
    for  $i = 0, 1, \dots, NSEC - 1$  do
       $qbuy[i] = \text{SUM}(qimq[i])$ 

    for  $i = 0, 1, \dots, NSEC - 1$  do
       $Government :: qbuy[i] = \frac{qpurch[i]}{pt[i]}$ 
       $Household :: qbuy[i] = nh * \frac{qsp[i]}{pt[i]}$ 

       $qinvtot = omegag[i] * Government :: qinv[i] + omegabl[i] * qinvbld + omegain[i] * qinvin +$ 
 $omega[i] * \text{SUM}(qinvlag)$ 
       $Bank :: qbuy[i] = \frac{qinvtot}{pt[i] * \frac{1 - txva2}{1 - txva1}}$ 

    for all externals do
       $qq = \text{INVERT\_MATRIX}(I - (io3 * \frac{1 - imp}{1 - x})) * \text{SUM}(qbuy * \frac{1 - imp}{1 - x})$ 
      for  $i = 0, 1, \dots, NSEC - 1$  do
         $qbuy[i] = io[i] * qq$ 

    for all sectors do
       $qtbody = \text{SUM}(qbuy)$ 
```

Sum across firms per market for $Market :: qbuy[i]$.

Matrix inversion requires the entire matrix intact. io3 is temporarily constructed from the externals' io.

The sum across qbuy is total buying from the current external.

qtbody is total buying from each sector.

Algorithm 44 Price Adjust

```
function MARKET::PRICE_ADJUST
   $ch = \frac{(1 - imp) * maxdp * pt}{4 * (marketiter - 1)}$ 
  if  $qtbody * (1 - imp) < \text{SUM}(qoptsudom)$  then
     $pt = pt - ch$ 
  else
     $pt = pt + ch$ 
```

Sum across firms per market.

5.5.4 Compute Imports

Algorithm 45 Compute Imports

```
function COMPUTE_IMPORTS
   $qimport = 0$ 
  for all markets do
     $qtbodyfor1 = imp * qtbody$ 
     $qtbodydom = qtbody - qtbodyfor1$ 
```

```

 $qmaxtsudom = \text{SUM}(0, \text{SUM}(qq + sto - minsto - qsufor))$ 
 $qtbuydom = \text{MIN}(qtbuydom, qmaxtsudom)$ 

 $qtbuyfor2 = qtbuy - (qtbuydom + qtbuyfor1)$ 
 $qtbuyfor = qtbuyfor1 + qtbuyfor2$ 

 $qimport = qimport + qtbuyfor * qpfor$ 

for all externals do
   $qtbuyfor = imp * qtbuy$ 
   $qtbuydom = qtbuy - qtbuyfor$ 

 $qimport = qimport + qtbuyfor * (1 - txva2) * qpdome * (1 + qdp)$ 

```

5.5.5 Domestic Results

Algorithm 46 Domestic Results

```

function DOMESTIC_RESULTS
  for all markets do
    MARKET::DOMESTIC_RESULTS
  for all externals do
    EXTERNAL::DOMESTIC_RESULTS

```

Algorithm 47 Domestic Results (Market)

```

function MARKET::DOMESTIC_RESULTS
   $qdpdom = \frac{pt - imp * qpfor}{(1 - imp) * qpdpdom} * (\frac{qpdpdom}{qpdpdom} - 1)$ 
   $qpdom = qpdom * (1 + qdpdom)$ 
   $qtsudom = qtbuydom$ 

```

Algorithm 48 Domestic Results (External)

```

function EXTERNAL::DOMESTIC_RESULTS
   $qdpdom = \frac{pt}{qpdom} - 1$ 
   $qpdom = qpdom * (1 + qdpdom)$ 

```

5.5.6 External sectors

Algorithm 49 External Sectors

```

function EXTERNAL_SECTORS
  for all externals do
     $qexportin = qq * x * qpdome * (1 - txva2)$ 
     $qsdome = qtbuydome * qpdome * (1 - txva2)$ 
     $qva = qsdome + qexportin - qbuy \times (qpdome * (1 - txva2))$ 

     $qexport = qexport + \text{SUM}(qexportin)$ 
     $qinpay = \text{SUM}(qinpay) - qinvbld - qinvin$ 

```

$qpdome * (1 - txva2)$ substitutes $qpfor$.

$qbuy$ is matrix multiplies with respective prices per sector.

5.5.7 Household Update

Algorithm 50 Household Update

```

function HOUSEHOLD::HOUSEHOLD_UPDATE
  for  $i = 0, 1, \dots, NEXPH - 1$  do
    if  $i \neq DUR$  and  $i \neq SAV$  then
       $qc[i] = qsp[i]$ 

       $stodur = \frac{pt[DUR]}{qp[DUR]} * stodur + qsp[DUR]$ 
       $qc[DUR] = rhodur * stodur$ 
       $stodur = stodur * (1 - rhodur)$ 

       $qsp[SAV] = qdi - \text{SUM}(qsp) + qsp[SAV]$ 
       $wh = wh + qsp[SAV]$ 
       $qsav = qsp[SAV] * nh$ 

    for  $i = 0, 1, \dots, NSEC - 1$  do
       $cva[i] = smooth[i] * cva[i] + (1 - smooth[i]) * \frac{qc[i]}{pt[i]}$ 

       $whra = smooth[SAV] * whra + (1 - smooth[SAV]) * \frac{wh}{qdi}$ 

    for  $i = 0, 1, \dots, NSEC - 1$  do
       $qp[i] = pt[i]$ 

       $oldqcpi = qcpi$ 
       $qcpi = \frac{\text{SUM}(qc)}{\text{SUM}(\frac{qc}{qp})}$ 
       $qdcpi = \frac{qcpi - oldqcpi}{oldqcpi}$ 

```

5.5.8 Indirect Taxes

Algorithm 51 Indirect Taxes

```
function INDIRECT_TAXES
  for all sectors do
     $qvatax = txva2 * (qpurch + nh * qsp)$ 
    if  $txva1 < 0$  then
       $more = \frac{qpdome*(1-txva2)}{1-txva1} * bank :: qbuy$ 
       $qvatax = qvatax + txva1 * more$ 
     $qvataximp = SUM(\frac{qtbuyfor}{qtbuy} * qvatax)$ 
```

qvatax, qpurch, qsp, and Bank::qbuy are indexed in the real code.

5.6 Inventory System

Algorithm 52 Sto System

```
function STO_SYSTEM
  FIRM_STO
  for all firms do
    FIRM::REFERENCE_INVENTORY_LEVELS
```

Algorithm 53 Firm Sto

```
function FIRM_STO
  for all firms do
    limsto = sto + qq - qsufor
    lower = MIN(limsto, minsto)
    upper = MIN(limsto, maxsto)

    qsubsdome = 0
    for all markets do
      totchsto = MIN(SUM(qq - qsufor) - qtsudome, SUM(upper - sto))
      qchsto = totchsto
      qwaste = SUM(qq - qsufor) - qtsudome - totchsto

      for all firms in market do
        if sto > upper then
          ch = upper - sto
        else
          ch = lower - sto
        sto = sto + ch
        qchsto = ch
        totchsto = totchsto - ch

    diff1 = SUM(upper - sto)
    diff2 = SUM(lower - sto)
    for all firms in market do
      if totchsto > 0 then
        ch =  $\frac{upper-sto}{diff1} * totchsto$ 
      else
        ch =  $\frac{lower-sto}{diff2} * totchsto$ 
      sto = sto + ch
      qchsto = qchsto + ch

    qsudome = qq - qsufor - qchsto
    qsdome = qsudome * (1 + rsubs) * qpdome * (1 - txva2)

    qsubsdome = qsubsdome + SUM(qsudome * rsubs * qpdome * (1 - txva2))
```

Sums across firms are per market.

Algorithm 54 Reference Inventory Levels

function FIRM::REFERENCE_INVENTORY_LEVELS

$j = \text{relative_quarter} + 1$

$$\text{curs} = 4 * \frac{s * (1 + \frac{\text{histds}}{4})^{j+1.5} + \text{cums} * (1 + \frac{\text{histds}}{4})^{\frac{j-1}{2}}}{j+4}$$

$$\text{curp} = \frac{4 * p * (1 + \frac{\text{histdp}}{4})^{j+1.5} + j * \text{cump} * (1 + \frac{\text{histdp}}{4})^{\frac{j-1}{2}}}{j+4}$$

$$\text{minsto} = \text{small} * \frac{\text{curs}}{\text{curp}}$$

$$\text{maxsto} = \text{big} * \frac{\text{curs}}{\text{curp}}$$

$$\text{optsto} = \text{minsto} + \text{beta} * (\text{maxsto} - \text{minsto})$$

for $i = 0, 1, \dots, NSEC - 1$ **do**

$$\text{minimsto}[i] = \text{imsmall} * \text{io}[i] * \text{share} * \frac{\text{curs}}{\text{curp}}$$

$$\text{maximsto}[i] = \text{imbig} * \text{io}[i] * \text{share} * \frac{\text{curs}}{\text{curp}}$$

$$\text{optimsto}[i] = \text{minimsto}[i] + \text{imbeta} * (\text{maximsto}[i] - \text{minimsto}[i])$$

io refers to the firm's market's io

5.7 Final Quarterly Prices, Sales, and Margin

Algorithm 55 Finalqpqsqm

function FIRM::FINALQPQSQM

$$qsu = qsu_{for} + qsu_{dom}$$

$$qds = \frac{qs_{for} + qs_{dom} - qs}{qs}$$

$$qs = qs_{for} + qs_{dom}$$

$$qdp = \frac{\frac{qs}{qsu} - qp}{qp}$$

$$qp = \frac{qs}{qsu}$$

$$qdva = qq * \frac{qp - \text{SUM}(io \times (qp_{dom} * (1 - txva2)))}{qva} - 1$$

$$qva = qva * (1 + qdva)$$

$$qsnet = qs - \text{SUM}(qimq \times (qp_{dom} * (1 - txva2)))$$

$$qm = 1 - \frac{l * \frac{qw}{4}}{qsnet}$$

io and qimq are matrix multiplied with respective prices.

5.8 Quarterly Cumulation

Algorithm 56 Quarterly Cum

function FIRM::QUARTERLY_CUM

$cuminv = cuminv + qinvlag$

$cumq = cumq + qq$

$cumva = cumva + qva$

$cums = cums + qs$

$cumsu = cumsu + qsu$

$cumsnet = cumsnet + qsnet$

$cumws = cumws + l * \frac{qw}{4}$

$j = relative_quarter + 1$

$cuml = \frac{(j-1)*cuml+l}{j}$

$cumm = 1 - \frac{cumws}{cumsnet}$

$cump = \frac{cums}{cumsu}$

5.9 Investment and Financing

Algorithm 57 Invfin

function FIRM::INVFIN

$$qrev = qm * qsnet + k2 * \frac{rik2}{4} - bw * \frac{rif}{4}$$

$$k1 = qinv + k1 * (1 + qdpk)$$

$$qdepr = rho * k1$$

$$k1 = k1 - qdepr$$

$$k1book = k1book + qinv$$

$$qdeprbook = \text{MAX}(0, \text{MIN}(qrev, rhobook * k1book))$$

$$k1book = k1book - qdeprbook$$

$$k3imed = \text{SUM}(imsto \times (qpdome * (1 - txva2)))$$

$$k3finish = sto * qp$$

$$k3 = k3imed + k3finish$$

$$qtax = txc * \text{MAX}(0, qrev - qdeprbook)$$

$$qdiv = rtd * qtax$$

$$qcash = 4 * (qtax + qdiv) + qs * rsubscash$$

$$qrr = 4 * \frac{qrev - qdepr}{k1 + k2 + k3}$$

$$qdeschk2 = rw * curs - k2$$

$$qdeschbw = \text{MAX}(0, 1 - elinv * (utref - \frac{qq}{qtop * wtix * (1 - res)})) * bw * (alfabw + betabw * (qdpk + \frac{qrr - rif}{4}))$$

$$qdeschbw = \text{MIN}(qdeschbw, redchbw * bw)$$

$$qdeschbw = qdeschbw * \text{FIRM::CHECK_BW}(\frac{bw + qdeschbw}{k1 + k2 + qdeschk2 + k3})$$

$$qinv = \text{FIRM::DELAY}$$

$$inveff = qtop * \frac{qp}{k1}$$

imsto is matrix multiplied with respective prices.

5.10 Government Accounting

Algorithm 58 Government Accounting

```

function GOVERNMENT::ACCOUNTING
     $j = \text{relative\_quarter} + 1$ 

     $qint = dep * \frac{ridepg}{4} - bw * \frac{ribwg}{4}$ 
     $qintfor = depfor * \frac{ridepgfor}{4} - bwfor * \frac{ribwgfor}{4}$ 
     $qws = l * \frac{qw}{4}$ 
     $qsubs = qsubsfor + qsubsdome + qsubscash$ 
     $qsp = qws + qsubs + qtrans + \text{SUM}(qpurch)$ 
     $qttax = qwtax + qitax + \text{SUM}(qvatax) + qctax$ 
     $qsurplus = qttax + qint + qintfor - qsp - qinv$ 
     $qmprint = \text{MAX}(0, \frac{money}{4} * \frac{\text{SUM}(s*ds)}{\text{SUM}(s)})$ 

     $posfor = posfor + qchposfor$ 
     $depfor = \text{MAX}(0, pos)$ 
     $bwfor = \text{MAX}(0, -posfor)$ 

     $qchpos = qsurplus + qmprint - qchposfor$ 
     $pos = pos + qchpos$ 
     $dep = \text{MAX}(0, pos)$ 
     $bw = \text{MAX}(0, -pos)$ 

     $cumwtax = cumwtax + qwtax$ 
     $cumitax = cumitax + qitax$ 
     $cumvatax = cumvatax + \text{SUM}(qvatax)$ 
     $cumctax = cumctax + qctax$ 
     $cumws = cumws + qws$ 
     $cuml = \frac{l+cuml*(j-1)}{j}$ 
     $cumsubs = cumsubs + qsubs$ 
     $cumtrans = cumtrans + qtrans$ 
    for  $i = 0, 1, \dots, NSEC - 1$  do
         $cumpurch[i] = qpurch[i]$ 
     $cuminv = cuminv + qinv$ 
     $cumint = cumint + qint + qintfor$ 
     $cummprint = cummprint + qmprint$ 

    if  $\text{relative\_quarter} = 3$  then
         $wtax = cumwtax$ 
         $itax = cumitax$ 
         $vatax = cumvatax$ 
         $ctax = cumctax$ 
         $dws = \frac{cumws}{ws} - 1$ 
         $ws = ws * (1 + dws)$ 
         $dw = \frac{cumws}{cuml*ws} - 1$ 
         $w = w * (1 + dw)$ 

```

```

subs = cumsubs
trans = cumtrans
for i = 0, 1, ..., NSEC - 1 do
    purch [i] = cumpurch [i]
sp = ws + subs + trans + SUM(purch)
inv = cuminv
intg = cumint
surplus = wtax + itax + vatax + ctax + intg - sp - inv
mprint = cummprint

```

posfor is a policy option.

5.11 Monetary Sector

Algorithm 59 Monetary Sector

```

function MONETARY_SECTOR
  BANK::BANK_TRANSACTIONS
  BANK::CREDIT_MARKET
  INVFIN_ADJUSTMENTS

```

Algorithm 60 Bank Transactions

```

function BANK::BANK_TRANSACTIONS
   $rfpay = \frac{ri - ribwfor}{maxridiff}$ 

   $qfasspay = \frac{fass + qexport}{1 + 4 * tmfass * (1 - rfpay)}$ 
   $qchfass = qexport - qfasspay$ 
   $fass = fass + qchfass$ 

   $qfdpay = \frac{fd + qimport}{1 + 4 * tmfd * (1 + rfpay)}$ 
   $qchfd = qimport - qfdpay$ 
   $fd = fd + qchfd$ 

   $qchliqbfor = qfasspay + Government :: qintfor - qfdpay - Government :: qchposfor$ 
   $liqbfor = liqbfor + qchliqbfor$ 

   $qchliqb = qintf + qsav + Government :: qchpos + qimport + Government :: qchposfor -$ 
   $qintk2 - Household :: qint - Government :: qint - qexport - Government :: qintfor$ 

```

Clamp rfpay between -lamda2 and lamda2.

Algorithm 61 Credit Market

```

function BANK::CREDIT_MARKET
   $qdemfund = \text{MAX}(0, -Government :: qchpos) + \text{SUM}(\text{MAX}(0, qdeschbw))$ 

   $qsupfund1 = \frac{liqb + qchliqb + \text{SUM}(qdeschk2) - rfund1 * (\text{SUM}(bw) + Government :: bw + \text{MAX}(0, -wh * nh))}{1 + rfund1}$ 
   $qsupfund2 = liqb + qchliqb + \text{SUM}(qdeschk2) - rfund2 * (\text{SUM}(k2 + qdeschk2) + Government ::$ 
   $dep + \text{MAX}(0, wh * nh))$ 
   $qsupfund = \text{MAX}(0, \text{MIN}(qsupfund1, qsupfund2))$ 

   $qsavhreq = \text{MIN}(kappa1 * \text{MAX}(0, qdi * nh), \text{MAX}(0, qdemfund - qsupfund))$ 
   $qredtbw = \text{MIN}(kappa2 * \text{SUM}(\text{MAX}(0, qdeschbw)), \text{MAX}(0, qdemfund - qsupfund - qsavhreq))$ 
  for all firms do
     $qchbw = qdeschbw - qredtbw * \frac{\text{MAX}(0, qdeschbw)}{\text{SUM}(\text{MAX}(0, qdeschbw))}$ 

   $qchri = lamda1 * \frac{qdemfund - qsupfund}{\text{MAX}(1, qdemfund)}$ 
   $ri = ri + qchri$ 

```

```

rif = ri
rih = ri − mb
rik2 = ri − mb
ribwg = ri
ridepg = ri − mb
ribwgfor = ribwfor
ridepgfor = ridepfor

```

Clamp *ri* between *minri* and *maxri*

Algorithm 62 Invin Adjustments

```

function INVFIN_ADJUSTMENTS
  oldinv = SUM(qinvlag)
  for all firms do
    bw = bw + qchbw
    qinvlag = MAX(0, qcash + qchbw − qdeschk2)
    qchk2 = qcash + qchbw − qinvlag
    k2 = k2 + qchk2

    nw = k1 + k2 + k3 − bw
    if nw < 0 then
      bad = bad + 1
    if bad ≥ 6 then
      NULLIFY_FIRM

  if no firms left in a market then
    exit simulation

  qtchbw = SUM(qchbw)
  qtchin = SUM(qinvlag) − oldinv
  qtchk2 = SUM(qchk2)

```

Algorithm 63 Bank Update

```

function BANK::BANK_UPDATE
  qchliqb = qchliqb + qtchk2 − qtchbw
  liqb = liqb + qchliqb

  assets = SUM(bw) + fass + liqbfor + liqb
  liabilities = SUM(k2) + wh * nh + Government :: pos + fd
  nw = assets − liabilities

  money = SUM(k2) + Government :: dep + wh * nh

```

5.12 National Accounting

Algorithm 64 National Accounting

function NATIONAL_ACCOUNTING

for all markets **do**

$qchtstocurf = \text{SUM}(k3finish) - tstocurf$

$tstocurf = tstocurf + qchtstocurf$

$qchtstocurm = \text{SUM}(sto * qpdom) - tstocurm$

$tstocurm = tstocurm + qchtstocurm$

$qgnpcur[id] = \text{SUM}(qsnet) + qchtstocurf$

▷ Explicit sector production

$qgnpfix[id] = pref * (\text{SUM}(qq) - qwaste)$

for all externals **do**

$qgnpcur[id] = cva$

▷ External sector production

$qgnpfix[id] = pref * qq$

$qgnpcur[10] = \text{SUM}(qvatax) + \text{SUM}(qchtstocurm - qchtstocurf) - qvataximp$ ▷ Indirect taxes

$qgnpcur[11] = -qsubs - qsubscash$

▷ Subsidies

$qgnpcur[12] = \text{Government} :: qws$

▷ Government wages

$qgnpcur[13] = \text{SUM}(qpurch)$

▷ Government purchases

$qgnpcur[14] = nh * \text{SUM}(qsp) - qsav$

▷ Private consumption

$qgnpcur[15] = \text{SUM}(qinvlag) - qtchin$

▷ Investments (explicit sectors)

$qgnpcur[16] = qinvin$

▷ Investments (external sectors, exclusive of housing)

$qgnpcur[17] = qinvbld$

▷ Investments (residential construction)

$qgnpcur[18] = \text{Government} :: qinv$

▷ Investments (government)

$qgnpcur[19] = \text{SUM}(qchtstocurm)$

▷ Inventory changes

$qgnpcur[20] = qexport$

▷ Exports

$qgnpcur[21] = -(qimport - qvataximp)$

▷ Imports

$qnpgfx[10] = \text{Government} :: l * \frac{wgre}{4}$

▷ Government wages deflated to reference

$qgnpfix[11] = \text{SUM}(pref * \frac{qpurch}{qpdom})$

▷ Government purchases

$qgnpfix[12] = \text{SUM}(pref * \frac{qsp*nh}{qpdom})$

▷ Private consumption (non-saving categories)

$qgnpfix[13] = \frac{\text{SUM}(pref*omega*(\text{SUM}(qinvlag)-qtchin))}{qpdom * \frac{1-txva2}{1-txva1}}$

▷ Investments (explicit sectors)

$qgnpfix[14] = \frac{\text{SUM}(pref*omegain*qinvin)}{qpdom * \frac{1-txva2}{1-txva1}}$

▷ Investments (external sectors, exclusive of housing)

$qgnpfix[15] = \frac{\text{SUM}(pref*omegabld*qinvbld)}{qpdom * \frac{1-txva2}{1-txva1}}$

▷ Investments (residential construction)

$qgnpfix[16] = \frac{\text{SUM}(pref*omegaeag*\text{Government}::qinv)}{qpdom * \frac{1-txva2}{1-txva1}}$

▷ Investments (government)

$qgnpfix[17] = \text{SUM}(pref * qchtsto)$

▷ Inventory changes

$qgnpfix[18] = \text{SUM}(pref * \text{SUM}(qsufor)) + \text{SUM}(pref * x * qq)$

▷ Exports

$qgnpfix[19] = -\text{SUM}(pref * qtbuyfor)$

▷ Imports

```

for  $i = 0, 1, \dots, \text{length of gnpcur}$  do
     $\text{cumgnpcur}[i] = \text{cumgnpcur}[i] + q\text{gnpcur}[i]$ 
for  $i = 0, 1, \dots, \text{length of gnpfix}$  do
     $\text{cumgnpfix}[i] = \text{cumgnpfix}[i] + q\text{gnpfix}[i]$ 
 $\text{cumexport} = \text{cumexport} + q\text{export}$ 
 $\text{cumimport} = \text{cumimport} + q\text{import}$ 

if  $\text{relative\_quarter} = 3$  then
     $\text{gnpcur} = \text{cumgnpcur}$ 
     $\text{gnpfix} = \text{cumgnpfix}$ 
     $\text{export} = \text{cumexport}$ 
     $\text{import} = \text{cumimport}$ 

```

Sum across firms per market for explicit sector production.

The first sum for fixed exports is across markets. The second is across externals.

6 Helper Functions

Algorithm 65 Qfr

```

function FIRM::QFR(l)
  return  $wtix * (1 - res) * qtop * (1 - e^{-\frac{tec}{qtop} * l})$ 

```

Algorithm 66 Rfq

```

function FIRM::RFQ(q)
  return  $\frac{qtop}{tec} * \text{LOG}(\frac{wtix * (1 - res) * qtop}{wtix * (1 - res) * qtop - q})$ 

```

Logarithm is base e.

Algorithm 67 Sat

```

function FIRM::SAT(q, l)
  if  $l > 0$  then
     $margin = 1 - \frac{l * \frac{qexpw}{4}}{q * qexpnet}$ 
  else
     $margin = 1 - \frac{\frac{qexpw}{4}}{wtix * (1 - res) * tec * qexpnet}$ 

  if  $margin \geq qtargm$  then
    return true
  else
    return false

```

Algorithm 68 Delay

```

function FIRM::DELAY
   $delta = \frac{1}{1 + \frac{4 * tminv}{3}}$ 

   $inv\_delay[2] = inv\_delay[2] + qinvlag$ 
   $inv\_delay[1] = inv\_delay[1] + delta * inv\_delay[2]$ 
   $inv\_delay[2] = inv\_delay[2] - delta * inv\_delay[2]$ 
   $inv\_delay[0] = inv\_delay[0] + delta * inv\_delay[1]$ 
   $inv\_delay[1] = inv\_delay[1] - delta * inv\_delay[1]$ 

   $ret = delta * inv\_delay[0]$ 
   $inv\_delay[0] = inv\_delay[0] * (1 - delta)$ 
  return ret

```

Algorithm 69 Bw Ratio

```
function FIRM::CHECK_BW(bw_ratio)
     $val = 1 - bw\_ratio$ 
    if  $val > 0.3$  then
        return 1
    else if  $val \leq 0.1$  then
        return 0
    else
        return  $0.0025 * (\text{FLOOR}(100 * val + 0.5) - 10)^2$ 
```

FLOOR rounds down to the nearest integer.

Algorithm 70 Solve

```
function FIRM::SOLVE
     $MAX\_ITERATIONS = 100$ 
     $TOLERANCE = 0.001$ 
     $b = \frac{qexpw}{(1-qtargm)*wtix*(1-res)*tec*qexpnet*4}$ 
     $y = \frac{1}{b}$ 

    for  $i = 0, 1, \dots, MAX\_ITERATIONS - 1$  do
         $f = b * y + e^{-y} - 1$ 
         $f\_prime = b - e^{-y}$ 
         $delta = \frac{f}{f\_prime}$ 
         $y = y - delta$ 

        if  $|delta| < TOLERANCE$  then
             $qplanl = y * \frac{qtop}{tec}$ 
             $qplanq = \text{QFR}(qplanl)$ 
            return
```

Newton-Rhapson method.

Algorithm 71 Nullify Firm

```
function NULLIFY_FIRM
     $lu = lu + l$ 
    remove firm from the container of firms
```

Algorithm 72 Choose Labour Target

function CHOOSE_LABOUR_TARGET $len = \text{length of } ll$ $\triangleright$ (firm labour concatenated to the unemployment pool) $scaled_unemployment = ll[len - 1] * skrepa$ $total_labour = \text{SUM}(ll) - ll[len - 1] + scaled_unemployment$ $rand_val = \text{random value between 0 and } total_labour$ $target = ll[len - 1]$ $\triangleright$ default to unemployment pool $cumsum = 0$ **for** $j = 0, 1, \dots, len - 1$ **do** **if** j is the unemployment pool **then** $cumsum = cumsum + scaled_unemployment$ **else** $cumsum = cumsum + ll[j]$ **if** $cumsum > rand_val$ **then** $target = j$ **break****return** $target$

List of Algorithms

1	Simulate	6
2	Simulate Year	6
3	Simulate Quarter	6
4	Yearly Init	7
5	Yearly Init (Government)	7
6	Yearly Init (Firm)	7
7	Yearly Exp	8
8	Yearly Targ	8
9	Yearly Update	9
10	Quarterly Exp	10
11	Quarterly Exp (Firm)	10
12	Quarterly Targ	10
13	Quarterly Exp (Market)	10
14	Quarterly Exp (External)	10
15	Prodplan	11
16	luupdate	11
17	luupdate (Labour)	11
18	Luupdate (Firm)	11
19	Prodfront	12
20	Prodfront (Firm)	12
21	Init Prodplan	12
22	Target Search	13
23	Plan Implies Recruitment	13
24	Try Reduce Production	13
25	Try Increase Production	14
26	Try Reduce Slack	14
27	Handle Layoffs	14
28	Labour Market	15
29	Glabour	15
30	Indlabour	15
31	Labour Search	15
32	Handle Successful Attack	16
33	Handle Labour Search Layoffs	16
34	Labour Update	17
35	Planqrevise	17
36	Export Market	18
37	Adjust Foreign	18
38	Domestic Market	19
39	Market Entrance	19
40	Init (Household)	19
41	Market Confront	20
42	Compute Expenditures	20
43	Compute Buying	21
44	Price Adjust	21
45	Compute Imports	21
46	Domestic Results	22

47	Domestic Results (Market)	22
48	Domestic Results (External)	22
49	External Sectors	23
50	Household Update	23
51	Indirect Taxes	24
52	Sto System	25
53	Firm Sto	25
54	Reference Inventory Levels	26
55	Finalqpqsm	27
56	Quarterly Cum	28
57	Invfin	29
58	Government Accounting	31
59	Monetary Sector	33
60	Bank Transactions	33
61	Credit Market	33
62	Invfin Adjustments	34
63	Bank Update	34
64	National Accounting	35
65	Qfr	37
66	Rfq	37
67	Sat	37
68	Delay	37
69	Bw Ratio	38
70	Solve	38
71	Nullify Firm	38
72	Choose Labour Target	39

7 Variable List

Simulation variables are categorised by class. *double* is the data type for real numbers. *int* is the data type for integers. *const* specifies the variable remains constant during simulation. Data types ending with (*init*) are initialised from csv files before simulation start. Square brackets indicates the variable is an array. For example, *ribwfor* is a *const double* [120]. This means the rate of interest on foreign borrowing is an array of 120 constant real numbers. In other words, the array is an exogenous input variable that controls the rate of interest on foreign borrowing for each simulation quarter.

Variable	Data Type	Description
Indexing constants		
DUR	const int	Index of the durables sector.
NSEC	const int	Number of sectors.
MAXT	const int	Max number of simulation quarters.
MAXY	const int	Max number of simulation years.
NEXPH	const int	Number of household expenditure categories.
NEXT	const int	Number of external sectors.
NGNPCUR	const int	Number of accounting values in gnpcur
NGNPFIX	const int	Number of accounting values in gnpfix
NMKT	const int	Number of market sectors.
SAV	const int	Index of savings.
Bank		
fass	double (init)	Holding of foreign trade credits (claims on foreign importers).
fd	double (init)	Sum of debts to foreign suppliers of Swedish imports.
kappa1	const double (init)	Maximum fraction of household's disposable income that can be stipulated as a reduction in household borrowing (increased net savings) in case of excess demand on the credit market.
kappa2	const double (init)	Maximum fractional reduction of firms' desired new borrowing in case of excess demand on the credit market.
lamda1	const double (init)	Used to compute the change in the rate of interest effected by supply-demand conditions in the credit market.
lamda2	const double (init)	Indicates how the foreign/domestic differential in the rate of interest impacts on average payment times for foreign trade credits.
liqb	double (init)	Holdings of "liquidity", of an unspecified nature.
liqbfor	double (init)	Holdings of foreign "liquidity", of an unspecified nature.
maxqchri	const double (init)	Limit on the rate of change for the rate of interest.
maxri	const double (init)	Ceiling on the rate-of-interest movements, used in the credit market.
maxridiff	const double (init)	Maximum difference between foreign and domestic rate of interest, which is allowed to have an impact on the rate of payment of foreign trade credits.

Variable	Data Type	Description
mb	const double (init)	Prescribed differential between domestic borrowing and lending rates of interest.
minri	const double (init)	Floor for rate-of-interest movements.
money	double	The stock of money in the economy, defined as total deposits in the bank.
nw	double (init)	Net value (assets - liabilities).
qbuy	double[NSEC]	Quarterly purchasing from each sector (volume terms).
qchfass	double	Quarterly change in the aggregate stock of trade credits to foreigners.
qchfd	double	Quarterly change in the aggregate stock of trade debts to foreigners.
qchliqb	double	Quarterly change in the bank's liquidity.
qchliqbfor	double	Quarterly change in foreign liquidity.
qchri	double (init)	Quarterly change in the base rate of interest.
qdemfund	double	Total quarterly demand on funds for new borrowing, from firms and the government.
qfasspay	double	Quarterly payment of trade credits, reducing the stock fd of debt to foreigners.
qfdpay	double	Quarterly payments of trade debts, reducing the stock fd of debt to foreigners.
qinvtot	double	Quarterly aggregate investments in the economy (money terms).
qsupfund	double	Availability of funds for new borrowing during one quarter, in concordance with prevailing monetary restrictions.
qsupfund1	double	Quarterly ceiling on the availability of funds for new borrowing, related to the necessity to have a liquidity in proportion to total lending.
qsupfund2	double	Quarterly ceiling on the availability of funds for new borrowing, related to the necessity to have a liquidity in proportion to total deposits in the bank.
qtchbw	double	Total quarterly new borrowing for all firms.
qtchinv	double	Total quarterly change in investments for all firms.
qtchk2	double	Total quarterly change in current assets for all firms.
qredtbw	double	Total quarterly amount firms must lower desired new borrowing, in the case of excess demand for funds.
rfpay	double	Accounts the effect of foreign/domestic rate-of-interest differentials on payment periods for foreign trade credits and debts.
rfund1	const double (init)	Fraction indicating the liquidity restriction on the bank as regards total lending.
rfund2	const double (init)	Fraction indicating the liquidity restriction on the bank as regards total deposits.
ri	double (init)	Base rate of interest, in parallel with which all other domestic rates move.

Variable	Data Type	Description
ribwfor	const double[MAXT] (init)	Exogenous foreign lending rate of interest.
ribwg	double	Rate of interest on government domestic borrowing.
ribwgfor	double	Rate of interest on government foreign borrowing.
ridepfor	const double[MAXT] (init)	Exogenous foreign deposit rate of interest.
ridepg	double	Rate of interest on government domestic deposits.
ridepgfor	double	Rate of interest on government foreign deposits.
rif	double	Rate of interest on firms' borrowing from the bank.
rih	double	Rate of interest on household savings.
rik2	double	Rate of interest on firms' deposits in the bank.
tmfass	const double (init)	Average payment period for foreign trade credits.
tmfd	const double (init)	Average payment period for foreign trade debts.
Firm		
aman	double[3] (init)	Vector accommodating the two-quarter lag of layoffs. The first component holds the number of people that can be fired this quarter, etc.
bad	double	Number of quarters the net worth has been negative.
big	const double (init)	Fraction of current yearly sales that firms consider as maximum for their finished-goods inventories.
bw	double (init)	Total borrowing.
chm	double (init)	Change in profit margin from one year to another.
cuminv	double	Yearly cumulation of investments.
cuml	double	Yearly cumulation of the number of employed.
cumm	double	Yearly cumulation of the profit margin.
cump	double	Yearly cumulation of the sales price.
cumq	double	Yearly cumulation of production volume.
cums	double	Yearly cumulation of sales value.
cumsnet	double	Yearly cumulation of net sales
cumsu	double	Yearly cumulation of sales volume.
cumva	double	Yearly cumulation of value added.
cumws	double	Yearly cumulation of the wage sum.
curp	double	Estimate of current yearly sales price, compatible with longer-term trends. (help variable)
curs	double	Estimate of current yearly sales, compatible with longer-term trends. (help variable)
dp	double (init)	Yearly change in sales price (a fraction).
dq	double	Yearly change in production volume (a fraction).
ds	double (init)	Yearly change in sales value (a fraction).
dva	double	Yearly change in value added (a fraction).
dw	double (init)	The yearly change in wage (a fraction).
exit	double	Discrepancy between actual and planned labour force (after market interactions). (help variable)
expdp	double	Yearly expected change in sales price (a fraction).
expds	double	Yearly expected change in sales (a fraction).

Variable	Data Type	Description
expdw	double	Yearly expected change in wage (a fraction).
expidp	double	Yearly "internally" expected change in sales price (a fraction).
expids	double	Yearly "internally" expected change in sales (a fraction).
expidw	double	Yearly "internally" expected change in wage (a fraction).
histdp	double (init)	Time-smoothed average of experienced yearly price increase.
histdpdev	double (init)	Time-smoothed average of the difference between actual and expected yearly price increase.
histdpdev2	double (init)	Time-smoothed average of the square of the difference between actual and expected yearly price increase.
histds	double (init)	Time-smoothed average of experienced yearly sales increase.
histdsdev	double (init)	Time-smoothed average of the difference between actual and expected yearly sales increase.
histdsdev2	double (init)	Time-smoothed average of the square of the difference between actual and expected yearly sales increase.
histdw	double (init)	Time-smoothed average of experienced yearly increase in wage level.
histdwdev	double (init)	Time-smoothed average of the difference between actual and expected yearly increase in wage level.
histdwdev2	double (init)	Time-smoothed average of the square of the difference between actual and expected yearly increase in wage level.
id	const int (init)	Identification number.
imbig	const double (init)	Fraction of one year's use of a certain input good, which a firm considers as the maximum inventory level for that good.
imsmall	const double (init)	Fraction of one year's use of a certain input good, which a firm considers as the minimum inventory level for that good.
imsto	double[NSEC] (init)	Inventory level for each kind of input good.
inv_delay	double[3]	Investment delay help variable used in DELAY.
ineff	double	Investment efficiency.
k1	double (init)	Replacement value of production equipment.
k1book	double (init)	Replacement value of production equipment (book).
k2	double (init)	Current assets.
k3	double	Value of total inventory.
k3imed	double	Value of input-goods inventory.
k3finish	double	Value of finished-goods inventory.
l	double (init)	Labour force.
layoff	double	Discrepancy between actual and planned labour force (before market interactions). (help variable)
m	double (init)	Profit margin.
market_id	const int (init)	Index of the sector the firm is part of.

Variable	Data Type	Description
maximsto	double[NSEC]	"Maximum" inventory levels of each kind of input good (volume terms).
maxsto	double	"Maximum" inventory level (volume terms).
mhist	double (init)	Average of past profit margins (a fraction).
minimsto	double[NSEC]	"Minimum" level for inventories of each kind of input good (volume terms).
minsto	double	"Minimum" inventory level (volume terms).
now	double	Number of people hired in one successful attack on the labour market. (help variable).
nw	double	Net value (assets - borrowing).
optimsto	double[NSEC]	Optimum inventory level for each kind of input good (volume terms).
optsto	double	"Optimum" level for finished-goods inventory (volume terms).
p	double (init)	Yearly average sales price.
q	double (init)	Yearly total production.
qcash	double	Net cash inflow during one quarter, before new borrowing and payments of investments.
qchbw	double	Quarterly change in borrowing.
qchk2	double	Quarterly change in current assets.
qchl	double	Quarterly labour force changes due to labour market interactions (retirements are not included).
qchqtop	double	Quarterly change in production capacity due to investments.
qchqtop1	double	Quarterly change in production capacity due to investments that can be used regardless of slack considerations.
qchqtop2	double	Quarterly change in production capacity due to investments (part that goes into slack).
qchw	double	Quarterly wage change in absolute terms.
qdepr	double	Quarterly depreciation of production equipment at replacement value.
qdeprbook	double	Quarterly depreciation of production equipment at book value (for taxation purposes).
qdeschbw	double	Desired new borrowing for one quarter.
qdeschk2	double	Desired change in current assets, to make these a certain proportion of sales.
qdiv	double	Quarterly dividend payments to the household aggregate.
qdp	double	Quarterly change in domestic price (a fraction).
qdpk	double	Quarterly relative price change for investment goods (equal for all firms).
qdq	double	Quarterly change in production volume (a fraction).
qds	double (init)	Quarterly change in sales value (a fraction)
qdva	double	Quarterly change in value added in current prices (a fraction).

Variable	Data Type	Description
qdw	double	Quarterly change in wage (a fraction).
qexpdp	double	Expected quarterly change in sales price (a fraction).
qexpds	double	Expected quarterly change in sales (a fraction).
qexpdw	double	Expected quarterly change in wage (a fraction).
qexp	double	Expected quarterly sales price.
qexpnet	double	Expected "net" sales price for the next quarter, with reductions for purchases of input goods.
qexpds	double	Expected quarterly change in sales value (a fraction).
qexps	double	Expected quarterly sales value.
qexpsu	double	Expected quarterly sales volume.
qexpdw	double	Expected quarterly change in wage.
qexpw	double	Expected quarterly wage.
qimq	double[NSEC]	Quarterly quantity (volume terms) bought of each kind of input-good.
qinv	double (init)	Quarterly installed investments (value terms).
qinvlag	double (init)	The money allocated for investments during one quarter, entering as demand on the product market next quarter.
qm	double	Quarterly profit margin.
qoptsu	double	Quarterly optimum sold volume.
qoptsudom	double	Quarterly optimum domestically sold volume.
qp	double (init)	Quarterly sales price.
qplanl	double	Quarterly planned labour force.
qplanq	double	Quarterly planned production volume.
qplanqsave	double	Help variable to save unmodified qplanq.
qq	double (init)	Quarterly production volume.
qrev	double	Quarterly gross revenue
qrr	double	Quarterly rate of return before taxes.
qs	double (init)	Quarterly sales value.
qsdom	double	Quarterly domestic sales value.
qsfor	double	Quarterly foreign sales value.
qsnet	double	Quarterly net sales value (less purchases of input-goods).
qsu	double	Quarterly sales volume.
qsudom	double	Quarterly domestic sales volume.
qsufor	double	Quarterly foreign sales volume.
qtargm	double	Quarterly target profit-margin.
qtax	double	Quarterly corporate tax.
qtop	double (init)	Quarterly potential output (units) at zero slack and infinite labour force.
qva	double (init)	Quarterly value added.
qw	double (init)	Quarterly wage level.
q2	double	Maximum quarterly production regarding sales plan and inventory maximum. (help variable)
q7	double	Quarterly production level, below which structural slack is activated. (help variable)

Variable	Data Type	Description
res	double (init)	Structural slack (fraction).
rw	const double (init)	Desired working capital (k2) as a fraction of yearly sales.
s	double (init)	Sales value.
sack	double	Quarterly number of people fired. (help variable)
share	const double (init)	Purchasing share of input-goods (used together with io).
small	const double (init)	Fraction of yearly sales considered as minimum for finished-goods inventories.
snet	double	Net sales value (less purchases of input goods).
sto	double (init)	Current inventory level (volume terms).
targm	double	Target profit-margin.
tec	double (init)	Technology factor (units per person and quarter)
va	double (init)	Value added.
w	double (init)	Wage level.
x	double (init)	Export share (fraction of sold volume).
Government		
bw	double	Domestic borrowing.
bwfor	double	Foreign borrowing.
ctax	double	Yearly aggregate corporate taxes.
cumctax	double	Yearly cumulative corporate taxes.
cumint	double	Yearly cumulative net interest receipts.
cuminv	double	Yearly cumulative investments.
cumitax	double	Yearly cumulative aggregate income taxes.
cuml	double	Yearly cumulative labour force.
cummprint	double	Yearly cumulative changes in the monetary base.
cumpurch	double	Yearly cumulative purchases of goods and services (less investments).
cumsubs	double	Yearly cumulative subsidies of the industry.
cumtrans	double	Yearly cumulative aggregate transfer payments to households.
cumvatax	double	Yearly cumulative aggregate value added tax.
cumws	double	Yearly cumulative wage sum.
cumwtax	double	Yearly cumulative aggregate payroll taxes.
dep	double	Domestic bank deposits.
depfor	double	Foreign bank deposits.
dw	double	Yearly change in wage (a fraction).
dws	double	Yearly change in wage sum (a fraction).
gkoff	const double[NSEC] (init)	Purchasing (less investments) in each sector, as a fraction of wage sum.
intg	double	Yearly net interest receipts
inv	double	Yearly Investments.
itax	double	Yearly aggregate income taxes.
l	double (init)	Labour force.
mprint	double	Yearly change in the monetary base (absolute value).
omega	const double[NSEC] (init)	Distribution vector of firms' outlays for investments allocated on purchases from each sector.

Variable	Data Type	Description
omegabld	const double[NSEC] (init)	Distribution vector of investments in residential construction allocated on purchases from each sector.
omegag	const double[NSEC] (init)	Distribution vector of government investments allocated on purchases from each sector.
omegain	const double[NSEC] (init)	Distribution vector of external sector investments allocated on purchases from each sector.
pos	double (init)	Net position in the bank.
posfor	double (init)	Net foreign deposit/borrowing position.
purch	double[NSEC]	Yearly purchases of goods and services (less investments) for each sector.
qbuy	double[NSEC]	Quarterly purchasing from each sector (volume terms).
qchl	double	Quarterly change in labour force.
qchpos	double	Quarterly change in net bank position.
qchposfor	double (init)	Quarterly change in net foreign deposit/borrowing position.
qchtxva1	const double[MAXT]	Quarterly change in value-added tax rate for investment goods. Entered exogenously.
qchtxva2	const double[MAXT]	Quarterly change in value-added tax rate for non-investment goods. Entered exogenously.
qctax	double	Quarterly aggregate corporate taxes.
qdw	double	Quarterly change in wage (a fraction).
qint	double	Quarterly net domestic interest receipts.
qintfor	double	Quarterly net foreign interest receipts.
qinv	double (init)	Quarterly investments. Entered exogenously.
qinvbld	double (init)	Quarterly aggregate investments in residential construction. Entered exogenously.
qinvin	double (init)	Quarterly aggregate investments by external sectors (less residential construction). Entered exogenously.
qitax	double	Quarterly aggregate income taxes.
qmprint	double	Quarterly change in the monetary base (absolute value).
qpurch	double[NSEC]	Quarterly purchases (money terms) of goods and services (less investments) from each sector.
qrealchl	double (init)	Quarterly net change in employment. Entered exogenously.
qsp	double	Total quarterly expenditures (less investments and interest payments).
qsubs	double	Total quarterly subsidy payments to firms.
qsubscash	double	Temporary quarterly subsidy payments to firms ("liquidity injections").
qsubsdom	double	Quarterly subsidy payments to firms (domestic sales).
qsubsfor	double	Quarterly subsidy payments to firms (exports).
qsurplus	double	Quarterly surplus.
qtrans	double	Quarterly transfer payments to households.
qttax	double (init)	Total quarterly tax receipts.
qvatax	double[NSEC]	Quarterly value added tax for each sector.

Variable	Data Type	Description
qvataximp	double	Quarterly part of total value added taxes which is related to imports.
qw	double (init)	Wage level.
qws	double	Quarterly wage sum.
qwtax	double	Quarterly aggregate payroll taxes.
rsubscash	const double (init)	Fraction used to compute temporary government subsidies to firms ("liquidity injections") in relation to sales value.
sp	double	Total yearly expenditures (less investments and interest payments).
subs	double	Total yearly subsidies to firms.
surplus	double	Yearly surplus.
trans	double	Yearly transfer payments to the household aggregate.
txc	const double[MAXY] (init)	Exogenous corporate-tax rate.
txil		const double[MAXY] (init)
txva1	const double (init)	Exogenous value-added-tax rate (investment goods).
txva2	const double (init)	Exogenous value-added-tax rate rate (non-investment goods).
txw	const double[MAXY] (init)	Exogenous payroll-tax rate for non-government sectors.
txwg	const double[MAXY] (init)	Exogenous payroll-tax rate for government sectors.
vatax	double	Total yearly value added tax.
w	double (init)	Yearly average wage.
wgref	const double (init)	Wage level for a reference year.
ws	double (init)	Total yearly wage sum.
wtax	double	Total yearly payroll taxes.
Household		
alfa3	const double (init)	Constant used to determined short-term swap between savings and expenditures on durables.
alfa4	const double (init)	Constant used to determined short-term swap between savings and expenditures on durables.
beta1	const double[NEXPH] (init)	Constant used to adjust household expenditures to the income constraint (All $\beta_1 \geq 0$).
beta2	const double[NEXPH] (init)	Constant used to adjust household expenditures to the income constraint ($\sum(\beta_2) = 1$).
beta3	const double[NEXPH] (init)	Constant used to adjust household expenditures to the income constraint ($\sum(\beta_3) = 0$).
cva	double[NSEC] (init)	Quarterly "addicted" consumption volume (units) in each expenditure category.
nh	const double (init)	Number of households.
pt	double[NSEC]	Offering price in each sector for each market iteration.
qbuy	double[NSEC]	Quarterly purchasing from each sector (volume terms).
qc	double[NSEC] (init)	Quarterly consumption in each non-saving expenditure category.

Variable	Data Type	Description
qchdcpi	double	Attempted change in consumer price index between quarters (a fraction).
qcp	double (init)	Consumer price index.
qdcpi	double (init)	Quarterly change in consumer price index (a fraction).
qdi	double	Quarterly disposable income.
qinpay	double (init)	Quarterly aggregate wage and capital income from external sectors.
qint	double	Quarterly aggregate interest receipts.
qp	double[NSEC] (init)	Quarterly domestic price in each expenditure category as households see them.
qprelcp	double	Preliminary consumer price index.
qsav	double	Quarterly aggregate savings.
qsavhreq	double (init)	Quarterly reduction in aggregate borrowing (forced increase in net savings) in the case of an excess demand for funds.
qsp	double[NEXPH]	Quarterly expenditure in each category.
qspe	double[NEXPH]	Quarterly "essential" expenditure in each category.
qspsavreq	double	Quarterly reduction in new borrowing (forced increase in new savings) in the case of excess demand for funds.
qtdiv	double (init)	Quarterly dividend payments from firms to households.
qti	double	Total quarterly income (before taxes).
qtws	double	Total quarterly wage sum in the economy (before payroll taxes).
rhodur	const double (init)	Depreciation rate of consumer durable goods (a fraction).
rtrans	const double (init)	Ratio between total transfer payments to households (less unemployment compensation) and total taxes.
smooth	const double[NEXPH] (init)	Constants used to time-smooth addicted consumption levels and the savings ratio.
stodur	double (init)	Stock of durable goods (value terms).
swap	double	Factor determining the short-term trade-off between savings and expenditures on consumer durables.
wh	double (init)	Wealth (current value of bank deposits).
whra	double (init)	Addicted wealth ratio (quotient between bank deposits and quarterly disposable income).
Labour		
entry	const double (init)	Exogenous parameter regulating the inflow of new persons to the labour market (fraction of total labour).
gamma	const double (init)	How big wage increase is needed for a person to leave a job for another.
ksi	const double (init)	How much a firm raises its wage level after an unsuccessful labour attack.
lf	double	Total labour force in the economy.
lu	double (init)	Number of people unemployed.
qchru	double	Quarterly change in rate of unemployment.

Variable	Data Type	Description
qdwind	double (init)	Average quarterly wage change in the industry (a fraction).
ret	const double (init)	Retirement rate on the labour market (a fraction).
rlu	const double (init)	Fraction used to compute unemployment compensation in proportion to average wage level in the industry.
ru	double (init)	Rate of unemployment (a fraction).
skrepa	const double (init)	Relative probability the unemployment pool is selected instead of firms during a labour market attack.
theta	const double (init)	Maximum fraction of a firm's labour force that it can lose in one labour market attack.
Moses		
bank	Bank	
cumexport	double	Yearly cumulative aggregate export value.
cumgnpcur	double[NGNPCUR]	Components of gross national product (current prices).
cumgnpfix	double[NGNPFIX]	Components of gross national product (fixed prices).
cumimport	double	Yearly cumulative aggregate import value.
export	double	Yearly aggregate export value.
externals	External[NEXT]	Container of the simulation's external sectors.
firms	Firm[]	Container of the simulation's firms.
gnpcur	double[NGNPCUR]	Yearly gross national product (current prices).
gnpfix	double[GNPFIX]	Yearly gross national product (fixed prices).
gov	Government	The simulation's Government class instance.
hh	Household	The simulation's Household class instance.
import	double	Yearly aggregate import value.
lab	Labour	The simulation's Labour class instance.
marketiter	const int (init)	Number of iterations on the domestic product market.
markets	Market[NMKT]	Container of the simulation's explicitly producing model sectors.
niteret.	const int (init)	Number of iterations on the labour mark
qexport	double	Quarterly export value.
qimport	double	Quarterly import value.
qintf	double	Quarterly interest payments from firms to the bank.
qintk2	double	Quarterly interest receipts by firms.
quarter	int	Current quarter in the simulation (0-indexed).
relative_quarter	int	Current quarter in the year (0-indexed).
year	int	Current year in the simulation (0-indexed).
Sector		
id	const int (init)	Sector index.
imp	double (init)	Import share.
io	const double[NSEC] (init)	Input-output coefficients. The amount of each input material a firm (or external sector) needs for production.
more	double	Additional value added tax on investments.
pref	const double (init)	Domestic prices from a reference year.
qbuy	double[NSEC]	Quarterly purchasing from each sector (volume terms).
qdpdom	double	Quarterly change in domestic price (a fraction).

Variable	Data Type	Description
qexpdpim	double	Expected quarterly change in the purchase price of the input good (a fraction).
qexppim	double	Expected quarterly purchase price of the input good.
qpdom	double (init)	Quarterly domestic price.
qpfor	double (init)	Quarterly foreign price.
qtbuy	double	Quarterly aggregate purchasing from this sector (including imports).
qtbuydom	double	Quarterly aggregate purchasing from this sector (excluding imports).
qtbuyfor	double	Quarterly aggregate imports within this sector.
tmimp	const double (init)	Time constant to adjust the import share.
Market		
maxdp	const double (init)	Maximum yearly domestic price deviation from firm expectations.
mtec	double (init)	Technology factor of modern equipment (potentially produced units per person per quarter).
qchtsto	double	Quarterly change in inventory to be distributed between firms.
qchtstocurf	double	Quarterly change in aggregate sector-level inventories, value at factor prices.
qchtstocurm	double	Quarterly change in aggregate sector-level inventories, value at market prices.
qdmtec	const double (init)	Quarterly rate of technology upgrade for production equipment (a fraction).
qdpfor	const double[MAXT] (init)	Quarterly change in foreign price (a fraction).
qmaxtsudom	double	Maximum quarterly sales volume due to minsto considerations. (help variable)
qprelpdom	double	The firms' initial offering price.
qtbuyfor1	double	Quarterly aggregate "normal" (related to an import fraction) imports.
qtbuyfor2	double	Quarterly aggregate "extra" (in case of large excess demand) imports.
qtsudom	double	Quarterly sold volume on the domestic market (exclusive of imports).
qwaste	double	Quarterly waste that occurs in case of insufficient finished-goods inventory capacities.
rsubs	const double (init)	Fraction used to compute government subsidies to firms in relation to their sales value.
tminv	const double (init)	Average delay time to install investments in new production equipment.
tmx	const double (init)	Time constant for firms to adjust their export share.
totchsto	double	Total change in finished-goods inventories that remains to be distributed between firms. (help variable).
tstocurf	double (init)	Aggregate finished-goods inventories (at factor prices).

Variable	Data Type	Description
tstocurm	double (init)	Aggregate finished-goods inventories (at market prices).
External		
qdp	const double[MAXT] (init)	Quarterly change in domestic price including value added tax (a fraction).
qexportin	double	Quarterly export value.
qq	double	Quarterly aggregate production volume.
qsdm	double	Quarterly domestic sales value.
qva	double	Quarterly value added.
x	const double (init)	Export fraction.